The Designer-Maker’s Agentic Design Playbook

The age of the ‘handoff’ is dead. AI and MCP have dissolved the barriers between describing, designing, and building a product. Agents can now directly manipulate our design tools, write our code, and launch our products. The technological walls have fallen, and the artificial walls separating Experience, Product, and Engineering must fall too.

This is the manual for the designer who intends to survive—and lead—what comes next.

For this to work you need to believe in:

1. **Files > Tools**
    - *Why?* Proprietary design tools trap your IP. You must be able to abandon your software without losing your work.
    - *Therefore:* Code is the ultimate source of truth, not artboards. MCP allows tools to act as temporary lenses for our code, rather than prisons for our data.
2. **Observations + consideration > Assumptions + speed**
    - *Why?* AI unlocks scale, but generating UI en mass wastes time, compute and users’ goodwill.
    - *Therefore:* Deep understanding of HCI, behavioural design, and cognitive science act as filters, turns AI scale into human value.
3. **Making Things > Making Pictures of Things**
    - *Why?* You cannot run analytics or accessibility test a picture with hotspots. Making pictures of software creates massive waste and is the antithesis of Lean UX.
    - *Therefore:* We prototype in functional code. The design canvas is an active workspace manipulated by Agents, not a museum for static mocks.

Design workflows

Creating

1. **Generate + curate:** Sketch by hand, then, in your design tool,[1] use AI agents to generate variations at *a scale that scares you*. Curate the results and help the agent learn by collaborating with it on ‘design language’ and ‘interaction guidelines’ specs, based on your prompts, constraints and feedback.
2. **Visual refinement:** Perform manual tweaks where it would be faster than reprompting.
3. **Canvas-to-code:** Use MCP to import your design using your chosen UI framework.[2]
4. **Version + vary:** Maintain multiple design variations in separate Git branches or subfolders in your IDE project folder for testing.

5. **Behaviour + flow:** Manually describe the user flow in BDD format, and prompt the Agent to turn these into test scripts for QA.
6. **Prototype development:** The Agent writes the logic and (via MCP) builds any tooling required to run the product (eg BaaS or mock data), then runs the test scripts to verify its own work before human review.
7. **Human polish:** Perform manual code tweaks for micro-interactions and extreme precision. Pair with a feasibility expert to ensure the prototype can be integrated into production with minimal rework.
8. **Cross-platform compilation:** Pair with a feasibility expert to ensure the prototype will integrate cleanly with production tooling.
9. **Add instrumentation:** Add monitoring and product analytics[3] into the code.
10. **Deploy:** Push the functional prototype to a live testing environment.

Iterating

1. **Code-first iteration:** Pair with a feasibility expert to refine, and use Git to branch, tweak code, test, and merge.
2. **Code-to-canvas:** To iterate visually, use Agents to push the coded flows directly back into the design tool.[4] Edit visually, and repeat the pipeline.

Remixing

1. **Source the truth:** Pull design system components from the codebase into your design tool via MCP.
2. **Create screens + flows:** Create or prompt new journeys using existing components.
3. **Define behaviour:** Write the BDD specifications for these new flows and generate new automated test scripts.
4. **Build + verify:** Bring the structural designs back into code. The Agent writes the connecting logic, runs the tests, and prepares it for deployment.

Who's involved?

This workflow is designed around 3 liminal roles, directed by a service owner. Because Agentic Workflows collapse traditional silos, these roles are explicitly hybrid.

1. The Solution engineer (Product manager / Engineer hybrid)

- *Focus:* Viability and Feasibility.
- *Key accountabilities:* Maps business goals to system architecture. Directs the Agent to connect to rapid BaaS for testing, or build custom infrastructure for scale. Manages the resulting technical debt and CI/CD pipelines.

2. The Product designer (Experience designer / Product manager hybrid)

- *Focus:* Viability and Desirability.
- *Key accountabilities:* Writes specifications that serve as business rules and Agent constraints. Orchestrates the telemetry strategy, and curates the high-level visual brand alignment on the canvas.

3. The Creative technologist (Experience designer / Engineer hybrid)

- *Focus:* Desirability and Feasibility.
- *Key accountabilities:* Operates MCP tools to sync the canvas and codebase. Prompts the Agent to write target-native code, then dives into the IDE to manually polish micro-interactions, physics, and accessibility.

The Traditional Triad Fallback

If organizational structure absolutely mandates it, this workflow can still function under the traditional product triad, provided all three adopt the “Maker” mindset:

- **A Product Manager (Viability):** Must learn to write logical specs to actively instruct the Agent.
- **An Experience Designer (Desirability):** Must master MCP orchestration, CLI navigation, and code-level polish.
- **A Software Engineer (Feasibility):** Must embrace the AI as a co-author, shifting from writing boilerplate to managing the structural integrity of the Agent’s output.

What this means for experience designers

Designers can no longer avoid being makers. Like all makers, to do so, they need to become intimately familiar with their “materials” so that they can work *with* the digital “grain”.

Top things to understand:

- **Agent architectures:** Prompting, guiding, and setting strict constraints for AI agents using the Model Context Protocol.
- **Mechanical sympathy:** Getting the most out of hardware, software and infrastructure by working with their quirks and constraints.
- **Declarative paradigms:** understanding how declarative frameworks render UI. This means grasping how to pass data and constraints through `props`, so you can dictate how the Agent structurally builds the app dynamically, rather than relying on hardcoded visual variants.
- **Data + APIs:** Mapping JSON structures and API responses to your component props so the Agent can successfully connect real-world data to the frontend.

- **Behavioural specification:** Writing logic and requirements to instruct Agents and generate test scripts.
- **The command line:** Spinning up local servers, managing packages, navigating the terminal and basic scripting.
- **Version control:** Using Git to manage user testing branches, resolving merge conflicts, and reviewing the Agent's Pull Requests.
- **Constraint definition:** Setting the boundaries for the Agent, including accessibility, localization, SEO and performance requirements.
- **Telemetry:** Defining what constitutes success for a hypothesis, and writing the specifications for what to track.
- **Debugging:** Reading logs, trace data flows, and reprompting the Agent toward the correct solution.
- **Deployment:** Managing CI/CD pipelines, sandboxing environments, and pushing code to modern hosting platforms.

---

1. Figma is fine for this but in future Paper is likely to be a better tool due to having a HTML canvas instead of a vector canvas. ↩
2. Ideally Dart/Flutter, because it can compile down to mobile, web, desktop and embedded experiences. ↩
3. eg Mixpanel, Hotjar, GTM etc ↩
4. Such as Figma's code-to-canvas skill, or preferably, Paper's MCP tools ↩